



Collège de
Maisonneuve

Formation continue

Programme LEA.8F (AEC)

Développement de logiciels : sécurité des applications mobiles, Web et de bureau

PLAN DE COURS

Services Web

Cours 420-941-MA | 90 heures (2-4-2)

Ete 2026

Construction et consommation d'APIs REST modernes avec
Node.js, React, Next.js, Prisma, PostgreSQL et Clerk

Chargé de cours : Ahmed Imed Eddine Rabah

Département : Informatique

Courriel : arabah@cmaisonneuve.qc.ca

Groupe : 25604

Préalable : Aucun

Table des matières

1	Présentation du cours	2
2	Compétences à développer	2
3	Contenu et déroulement du cours	2
4	Activités d'enseignement et d'apprentissage	5
5	Évaluation formative et sommative	5
5.1	Grille d'évaluation	5
5.2	Détail des évaluations sommatives	6
5.3	Ateliers pratiques	6
5.4	Laboratoires	6
5.5	Examen intra – Hackathon de mi-session (48h)	7
5.6	Épreuve finale – Projet intégrateur	7
6	Modalités d'application des politiques institutionnelles et règles départementales	7
6.1	Double seuil de réussite	8
6.2	Présence aux évaluations	8
6.3	Reprise d'une évaluation	8
6.4	Remise des travaux et pénalités de retard	8
6.5	Plagiat, fraude et utilisation des assistants d'IA	8
6.6	Qualité du français	8
7	Recours prévus pour les étudiants	9
8	Médiagraphie	9
8.1	Documentation officielle (à consulter en priorité)	9
8.2	Sécurité	9
8.3	Manuels recommandés	9
8.4	Ouvrages et ressources facultatifs	9
9	Frais et plateforme	9
10	Disponibilité	10

1. Présentation du cours

Le cours **Services Web** couvre les composants logiciels qui gèrent les interactions des programmes distants communiquant sur le Web. À la fin du cours, l'étudiant.e doit être en mesure de **décrire l'interfonctionnement des protocoles Web**, de **concevoir et développer des APIs Web**, des services Web ainsi que le code client qui interagit avec ces services. L'étudiant.e doit pouvoir préparer l'environnement de développement, déployer et documenter les services Web, et contrôler la qualité du code produit.

Ce cours est offert à la **quatrième étape** et relève du champ de compétence *Effectuer le développement de services d'échange de données*. Il complète la formation en programmation Web en étendant les compétences côté serveur à la conception, à la mise en œuvre et à la **sécurisation** des services Web, y compris les communications de processus à processus. Aucun préalable n'est nécessaire et ce cours est préalable au *420-951-MA Applications Web Transactionnelles*.

Objectif global

Au terme du cours, l'étudiant.e sera capable de **concevoir, développer, sécuriser, tester, documenter et déployer** un service web REST complet en utilisant une stack moderne JavaScript/TypeScript (**Node.js, React, Next.js, Prisma, PostgreSQL, Clerk**), tout en respectant les bonnes pratiques de l'industrie (OWASP, OpenAPI, CI/CD).

2. Compétences à développer

Énoncé	Éléments de compétence
00SV Effectuer le développement de services d'échange de données (90H)	1. Analyser le projet de développement de l'application. 2. Préparer l'environnement de développement informatique. 3. Préparer la base de données. 4. Programmer la logique d'application pour le service. 5. Programmer une application de mise à l'essai utilisant le service. 6. Contrôler la qualité du service. 7. Participer au déploiement du service. 8. Produire la documentation.

3. Contenu et déroulement du cours

Le calendrier ci-dessous peut être ajusté en cours de session selon le rythme d'apprentissage du groupe. Les heures hors classe sont dédiées aux ateliers en cours et à la progression du projet fil rouge.

Semaine	Objectifs d'apprentissage	Laboratoires / activités
S1	Introduction aux services Web — Définir un service web et son rôle dans une architecture — Protocole HTTP : verbes, statuts, en-têtes — Comparer REST, GraphQL et SOAP — Outils : Postman, navigateur, Dev-Tools	Installation : VS Code, Node.js LTS, Postman, Git Création d'un projet Next.js + TypeScript Présentation du projet fil rouge
S2	Architecture REST et API Routes — Principes REST : ressources, stateless, CRUD, HATEOAS — Créer une route GET avec Next.js (App Router) — Bonnes pratiques de nommage d'endpoints	Exercice 1 : <code>/api/posts</code> avec retour JSON Exercice 2 : Interface React qui consomme l'API via <code>fetch()</code>
S3	Requêtes POST / PUT / DELETE et formats — Manipuler les méthodes mutatives — Transmettre des données (JSON, FormData) — Validation des entrées avec Zod	Atelier 1 : Formulaire + ajout de données (POST) Structure RESTful respectée Début du Laboratoire #1
S4	Structuration et typage des données — Concevoir des contrats d'API typés — Définir des types/interfaces TypeScript partagés — Pattern <i>DTO</i> et séparation des couches	Atelier 2 : Organisation des dossiers, typage strict, séparation contrôleur/service
S5	Base de données et persistance (PostgreSQL + Prisma) — Installation et configuration de PostgreSQL — Modélisation relationnelle de base — Intégration de Prisma : <code>schema.prisma</code>, migrations, seed	Atelier 3 : Connexion à PostgreSQL via Prisma + premier modèle persisté Remise du Laboratoire #1
S6	Authentification avec Clerk — Intégration de Clerk dans un projet Next.js — Composants <code><SignIn/></code>, <code><SignUp/></code>, <code><UserButton/></code> — Récupération de l'utilisateur courant côté serveur (<code>auth()</code>)	Exercice : page de connexion Clerk + accès conditionnel à <code>/dashboard</code> Début du Laboratoire #2

Semaine	Objectifs d'apprentissage	Laboratoires / activités
S7	Middleware, sécurité et OWASP API — Middleware Next.js et protection de routes (<code>clerkMiddleware</code>) — OWASP API Top 10 (injection, BOLA, rate limiting) — CORS, en-têtes de sécurité (<i>helmet-like</i>)	Atelier 4 : Protection des routes API avec Clerk + limitation de débit (<i>rate limit</i>)
S8	Gestion des erreurs HTTP et observabilité — Codes 4xx / 5xx appropriés — Format d'erreur normalisé (RFC 7807) — Journalisation structurée et traçabilité	Atelier 5 : Simulation et gestion d'erreurs API, journal structuré
S9	Consommation d'APIs externes — Consommer un service tiers depuis un Server Component — Gestion des clés d'API et variables d'environnement — Sécuriser les appels sortants (HTTPS, en-têtes)	Atelier 6 : Intégration d'une API publique (ex. OpenWeather ou TheMovieDB) dans une page Next.js Remise du Laboratoire #2
S10	Hackathon de mi-session (48h) — Concevoir et livrer un mini-service web complet — Travail en équipe, gestion du temps et du périmètre — Démo finale devant la classe	Examen intra – Hackathon de 48 heures Lancement le vendredi 18h, remise le dimanche 18h Présentations en classe le lundi suivant
S11	Webhooks et traitements asynchrones — Réagir aux événements HTTP entrants — Vérification de signature de webhook — Idempotence et fiabilité des traitements	Atelier 7 : Webhook Clerk <code>/api/webhooks/clerk</code> qui synchronise les utilisateurs dans PostgreSQL Début du Laboratoire #3
S12	Tests automatisés et validation — Tests unitaires (Vitest / Jest) — Tests d'intégration d'API (Supertest) — Tests manuels documentés avec Postman	Atelier 8 : Suite de tests pour <code>/api/posts</code> et routes protégées par Clerk

Semaine	Objectifs d'apprentissage	Laboratoires / activités
S13	Documentation d'un service web — OpenAPI 3.x / Swagger UI — Documentation Markdown lisible — Exemples de requêtes et de réponses	Atelier 9 : Génération de API_DOC.md et fichier openapi.yaml
S14	Déploiement et CI/CD — Variables d'environnement et secrets — Déploiement continu sur Vercel — Pipeline GitHub Actions (lint + test)	Atelier 10 : Déploiement Vercel + pipeline CI minimal Remise du Laboratoire #3
S15	Épreuve finale – Projet intégrateur — Démonstration du service web — Justification des choix techniques — Présentation orale (15 min) + questions	Remise du projet intégrateur, rapport et présentation orale

4. Activités d'enseignement et d'apprentissage

Les principales méthodes pédagogiques sont :

- Exposés théoriques en alternance avec des évaluations formatives (exercices) et sommatives (laboratoires, hackathon, projet intégrateur).
- Ateliers et travaux pratiques réalisés principalement en classe, complétés hors classe au besoin.
- Démonstrations en direct (*live coding*) et études de cas tirées de l'industrie.
- Travail en équipe (3 personnes maximum) sur les laboratoires et lors du hackathon de mi-session.
- Revue de code par les pairs (*peer review*) avant la remise du projet intégrateur.

Outils d'IA et assistance au code. L'utilisation d'assistants comme Claude, ChatGPT, Codex ou GitHub Copilot est **permise et même encouragée** dans les ateliers et le projet intégrateur, à condition que l'étudiant.e **comprenne, puisse expliquer et défendre** chaque ligne livrée. Lors des examens et du hackathon, leur utilisation peut être restreinte selon les consignes spécifiques de l'évaluation.

5. Évaluation formative et sommative

L'**évaluation formative** se fait en continu par la réalisation d'exercices pratiques en classe, avec le soutien du professeur. La réalisation de tous les exercices formatifs est *fortement recommandée*.

L'**évaluation sommative** s'effectue au moyen de **trois laboratoires**, d'un **hackathon de mi-session** et d'une **épreuve finale** sous forme de projet intégrateur.

5.1 Grille d'évaluation

Catégorie	Pondération
Théorie (Laboratoire #1)	10%
Laboratoires (#2 et #3)	40%
Examen de mi-session (Hackathon 48h)	10%
Projet intégrateur (Épreuve finale)	40%
TOTAL	100%

5.2 Détail des évaluations sommatives

Outil	Description	Pond.	Échéancier
Laboratoire #1	Évaluation théorique : protocoles, REST, premières routes API	10%	Semaines 3 à 5
Laboratoire #2	Partie 1 pratique du projet : BD, auth, sécurité	20%	Semaines 6 à 9
Hackathon 48h	Examen intra en équipe : mini-service web complet livré en 48 heures, présentation en classe	10%	Semaine 10 (après le Lab #2)
Laboratoire #3	Partie 2 pratique du projet : web-hooks, tests, déploiement	20%	Semaines 11 à 14
Épreuve finale	Remise du projet intégrateur, rapport PDF et présentation orale (15 min)	40%	Bloc de 3 heures de la semaine 15

5.3 Ateliers pratiques

Les ateliers sont des travaux pratiques de petite envergure réalisés en classe et terminés hors classe au besoin. Ils sont **formatifs** et guidés par le professeur. Ils servent à préparer les laboratoires et le projet intégrateur.

5.4 Laboratoires

Les laboratoires sont des travaux pratiques **évalués**. Ils sont réalisés individuellement ou en équipe de **trois personnes maximum**. Chaque laboratoire fait l'objet d'une grille de correction détaillée distribuée au moment de l'énoncé.

5.5 Examen intra – Hackathon de mi-session (48h)

Hackathon de mi-session (48h)

L'examen intra prend la forme d'un **hackathon de 48 heures** en équipe (3 personnes max.), réalisé après la remise du Laboratoire #2. Il remplace l'examen traditionnel pour mieux refléter les conditions réelles de l'industrie.

Format :

- **Lancement** : vendredi soir, présentation du défi et des contraintes techniques.
- **Réalisation** : 48 heures pour livrer un service web fonctionnel répondant à un cahier des charges remis sur place.
- **Remise** : dimanche soir, dépôt du code sur GitHub + court rapport en Markdown.
- **Présentation** : démonstration de 10 minutes en classe le lundi suivant, suivie de questions.

Critères d'évaluation : fonctionnalité (40%), qualité du code et architecture (20%), sécurité (15%), documentation (10%), créativité et complétude (15%).

Outils : la documentation officielle, la stack vue en classe et les assistants d'IA sont permis. La compréhension du code livré sera vérifiée durant la présentation.

5.6 Épreuve finale – Projet intégrateur

À travers un projet intégrateur, l'étudiant.e devra démontrer la maîtrise de la **création, consommation, test, sécurisation** et **documentation** d'un service web en utilisant la stack vue en classe (**Node.js / React / Next.js / Prisma / PostgreSQL / Clerk**) ou une stack équivalente approuvée. Du temps est alloué aux semaines 13 et 14 pour faire progresser le projet.

Livrables attendus :

- Un **rapport au format PDF** détaillant le service web développé (apprentissages, réalisations, défis, choix techniques).
- Une **présentation orale de 15 minutes** avec démonstration en direct.
- Le **code source** déposé sur GitHub (lien fourni dans le rapport).
- Une **documentation OpenAPI ou Markdown** décrivant les endpoints.
- Un **déploiement fonctionnel** (Vercel, Railway ou équivalent).

Un énoncé détaillé du projet intégrateur sera distribué au plus tard à la **huitième semaine**. Sous réserve de l'approbation du professeur, deux équipes ne peuvent choisir le même sujet et le sujet ne peut avoir été couvert dans un autre cours de la formation.

6. Modalités d'application des politiques institutionnelles et règles départementales

6.1 Double seuil de réussite

Les évaluations sommatives assujetties à l'atteinte du **double seuil de réussite** du cours sont :

- Le **hackathon de mi-session** (examen intra) ;
- L'**épreuve finale** (projet intégrateur).

L'étudiant.e doit obtenir une moyenne pondérée d'au moins **50%** sur ces deux évaluations contrôlées, en plus d'une moyenne générale d'au moins **60%** pour l'ensemble des sommatives. À défaut, la note au bulletin ne peut excéder 49%.

6.2 Présence aux évaluations

La présence est obligatoire aux évaluations sommatives. Toute absence doit être justifiée de façon satisfaisante au professeur, sinon la note 0 est attribuée. Pour les fêtes religieuses, l'étudiant.e doit aviser par écrit avant la fin de la deuxième semaine de la session (PIEA, point 4.4).

6.3 Reprise d'une évaluation

Aucune reprise possible sauf en cas d'absence justifiée. Dans le cas d'une absence justifiée à l'un des volets de l'épreuve finale, cette évaluation *doit* être reprise. Aucune évaluation sommative ne peut être reprise oralement.

6.4 Remise des travaux et pénalités de retard

Si l'heure de remise n'est pas précisée, le travail doit être remis avant le début des cours du jour ouvrable suivant. Lorsque le professeur accepte un retard, une pénalité de **10% de la note maximale par jour ouvrable** s'applique, jusqu'à concurrence de 50%. Aucun travail n'est accepté au-delà de 5 jours ouvrables de retard.

6.5 Plagiat, fraude et utilisation des assistants d'IA

Les points de l'article 4.9 de la PIEA s'appliquent. Spécifiquement pour ce cours :

- Toute portion de code générée par un assistant d'IA doit être **comprise et défendable** par l'étudiant.e lors d'une vérification orale.
- La copie intégrale de code provenant d'un.e autre étudiant.e ou d'un dépôt public, sans citation et sans adaptation, est considérée comme du plagiat.
- Lors du hackathon, l'utilisation des assistants d'IA est permise, mais chaque membre de l'équipe doit pouvoir expliquer toute partie du livrable.

6.6 Qualité du français

Un professeur peut refuser un travail dont la langue, la présentation ou la lisibilité sont jugées insuffisantes. L'étudiant.e devra alors reprendre le travail et subir les pénalités de retard applicables.

7. Recours prévus pour les étudiants

Voir la section *Extraits des politiques institutionnelles et départementales* dans le guide *Étudier à Maisonneuve*. En cas de recours, l'étudiant.e peut s'adresser au responsable de programme à la Formation continue.

8. Médiagraphie

8.1 Documentation officielle (à consulter en priorité)

- **Node.js** : <https://nodejs.org/fr/docs>
- **React** : <https://react.dev>
- **Next.js** : <https://nextjs.org/docs>
- **TypeScript Handbook** : <https://www.typescriptlang.org/docs/handbook>
- **Prisma** : <https://www.prisma.io/docs>
- **PostgreSQL** : <https://www.postgresql.org/docs/>
- **Clerk (auth)** : <https://clerk.com/docs>
- **Zod (validation)** : <https://zod.dev>
- **Vercel** : <https://vercel.com/docs>
- **OpenAPI 3.x** : <https://swagger.io/specification/>

8.2 Sécurité

- **OWASP API Security Top 10** : <https://owasp.org/API-Security/>
- **MDN – HTTP** : <https://developer.mozilla.org/fr/docs/Web/HTTP>

8.3 Manuels recommandés

- GRINBERG, Miguel (2024). *REST APIs with Flask and Python*, 2^e éd. – pour comparer les approches.
- BRUNS, Mathias (2023). *Real-World Next.js*, Packt Publishing.
- BANKS, Alex & PORCELLO, Eve (2024). *Learning React*, 3^e éd., O'Reilly.

8.4 Ouvrages et ressources facultatifs

- DUMINIL, Nicolas (2019). *AWS – Gérez votre infrastructure sur la plateforme cloud d'Amazon*.
- *The Twelve-Factor App* : <https://12factor.net/fr/> (méthodologie de référence pour les services web modernes).
- *web.dev* (performance et bonnes pratiques) : <https://web.dev>

9. Frais et plateforme

Aucun frais d'inscription à une plateforme tierce n'est exigé pour ce cours. Les comptes suivants, tous gratuits, sont toutefois requis pour suivre les ateliers et réaliser le projet :

- **GitHub** (<https://github.com>) – dépôt du code des laboratoires et du projet intégrateur.

- **Vercel** (<https://vercel.com>) – déploiement continu des applications Next.js à partir de la semaine 14.
- **Clerk** (<https://clerk.com>) – service d’authentification à partir de la semaine 6.
- **PostgreSQL** – instance locale, ou hébergement gratuit chez **Neon**, **Supabase** ou **Railway**, utilisé à partir de la semaine 5.

Tous ces services offrent un palier gratuit suffisant pour les besoins du cours.

10. Disponibilité

Sur demande par MIO ou par courriel à arabah@cmaisonneuve.qc.ca. Une réponse est généralement fournie dans un délai de **48 heures ouvrables**.

★ **Bonne session !**

Construisez, cassez, corrigez, documentez et déployez.
C’est en livrant qu’on devient développeur.